

Neural Networks

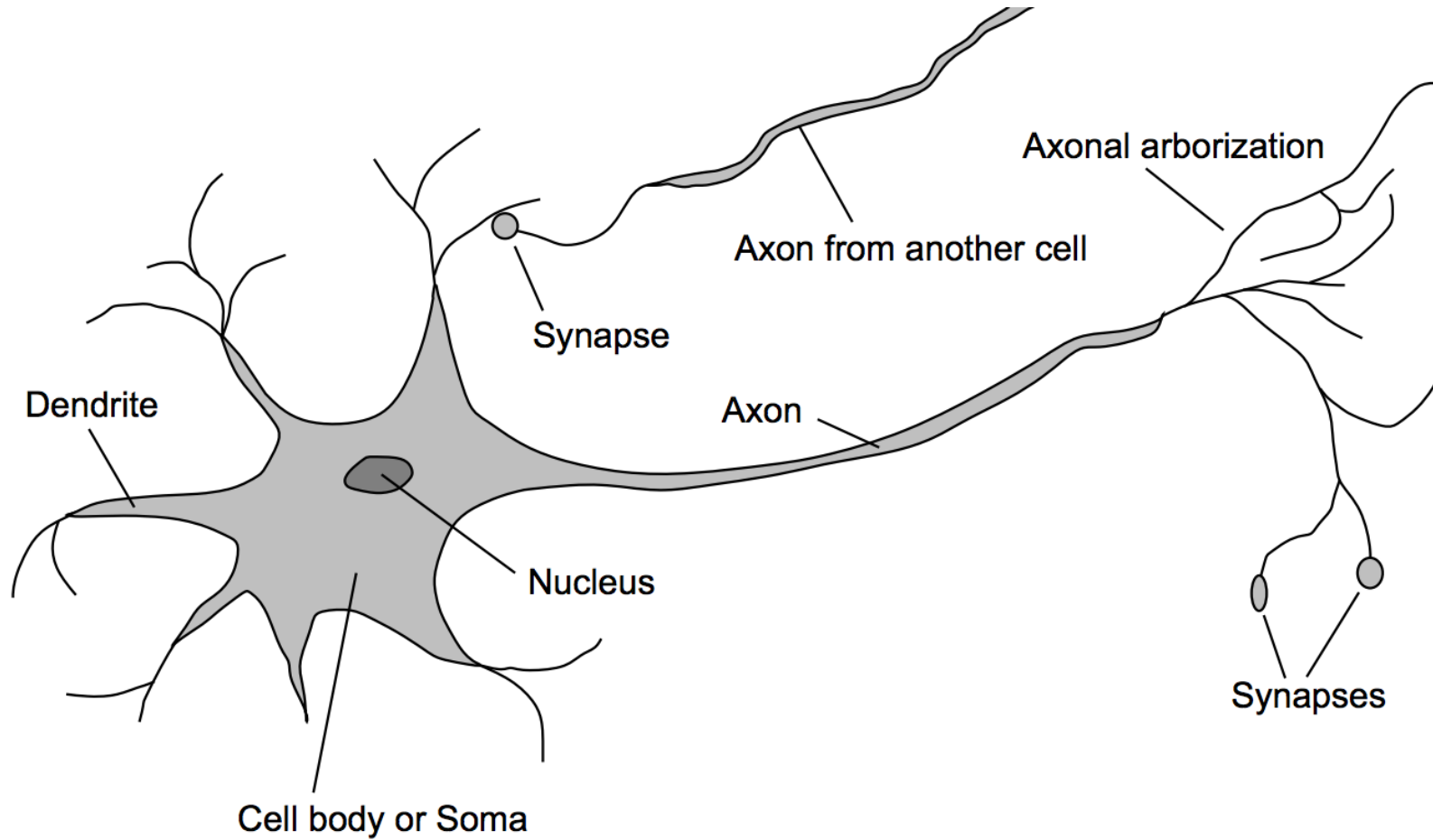
Enora Rice

09/24/26

Today

- Neural networks (Chapter 6)
 - Structure
 - Basic forward inference
 - Common NLP use cases for NNs
 - Text classification
 - Language models

These are in your brain



Biologically Inspired Unit

This is not in your brain

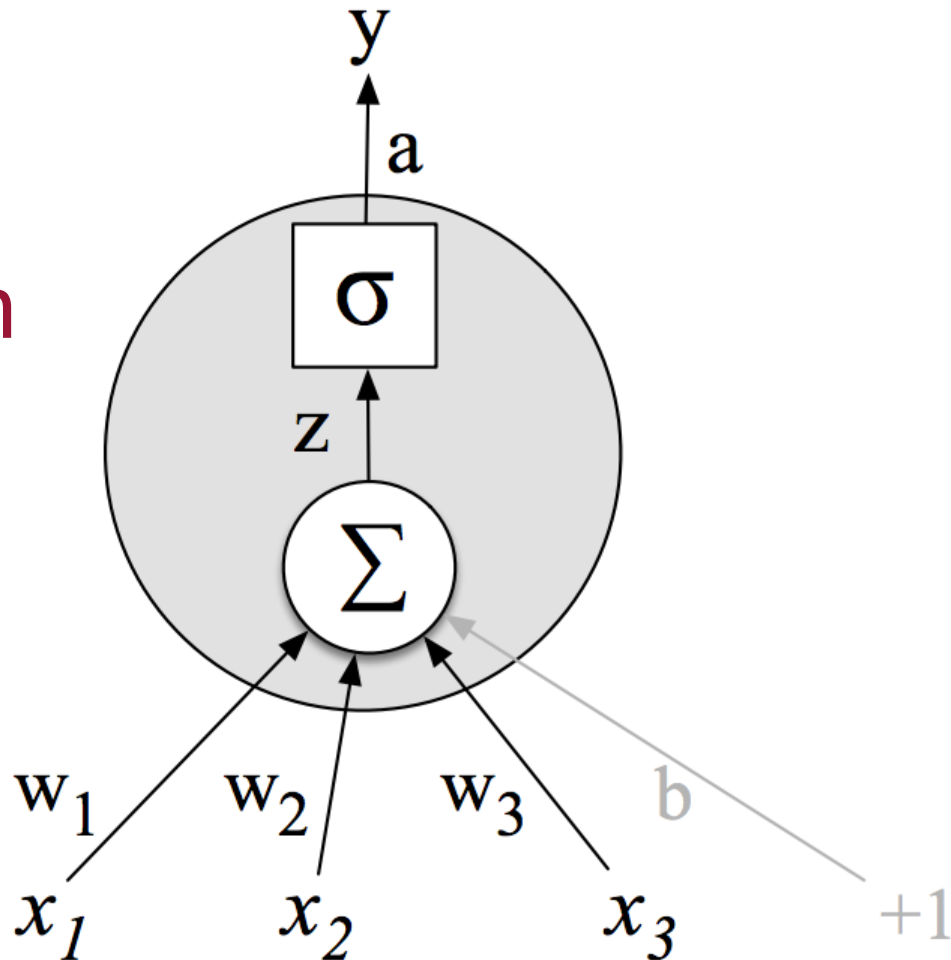
Activation value

Non-linear transform

Weighted sum

Weights

Input layer



Perceptron

- Basic neural unit
- Contains a binary output
- It is a *linear* classifier

Our input will be a vector $\mathbf{x}$, and our label will be y

$$\mathbf{x} = [x_1 \dots x_n]$$
$$\mathbf{y} \in \{\mathbf{0}, \mathbf{1}\}$$

$$z = b + \sum_{i=1}^n w_i x_i$$

$$\hat{y} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

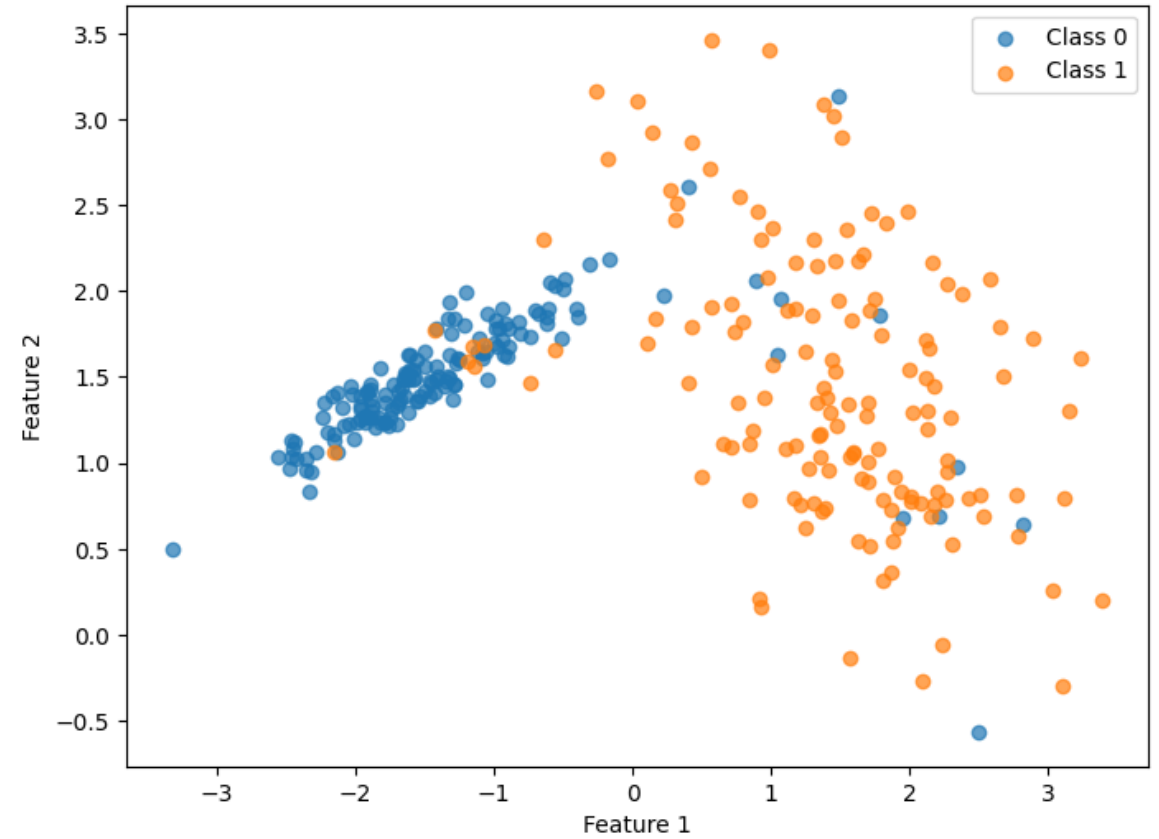
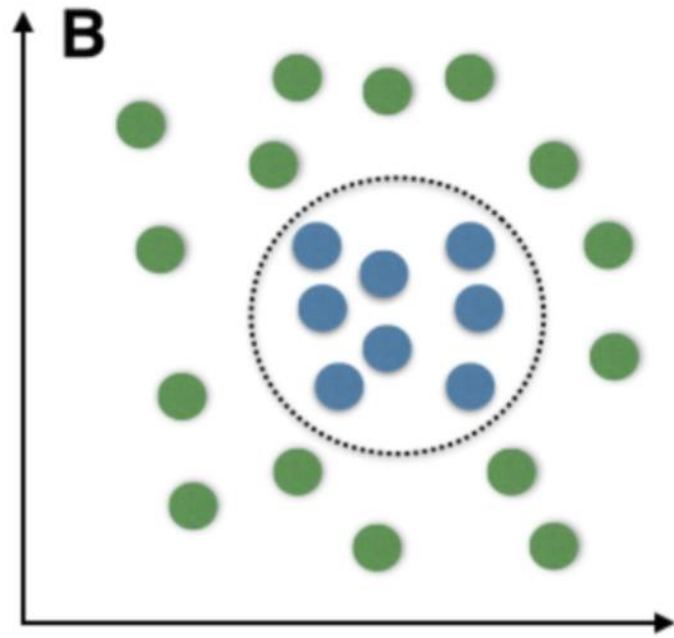
Perceptron

Our predicted value is simply a dot product between our weights $\mathbf{w}$ and our input example $\mathbf{x}$:

$$\mathbf{w} = \begin{bmatrix} b = w_0 \\ w_1 \\ \dots \\ w_n \end{bmatrix} \text{ and } \mathbf{x} = \begin{bmatrix} x_0 = 1 \\ x_1 \\ \dots \\ x_n \end{bmatrix}$$

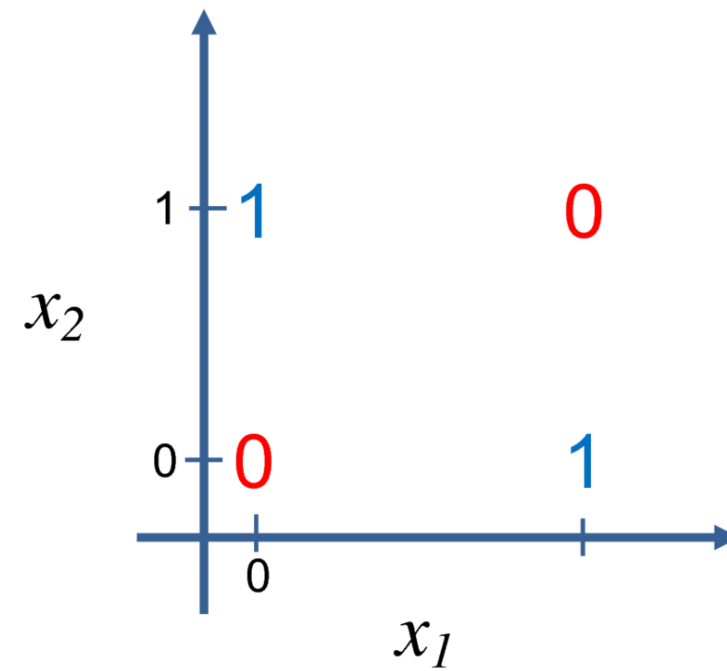
$$\mathbf{z} = \mathbf{w} \cdot \mathbf{x} = \sum_{i=0}^n w_i * x_i \text{ and } \hat{y} = \begin{cases} 1 & \text{if } \mathbf{z} > 0 \\ 0 & \text{if } \mathbf{z} \leq 0 \end{cases}$$

Non-Linearly Separable Data



XOR Problem

$A \blacklozenge$	$B \blacklozenge$	$A \oplus B$
F	F	F
F	T	T
T	F	T
T	T	F



XOR Problem

XOR has a non-linear decision boundary

XOR can be written as: $\neg(A \wedge B) \wedge (A \vee B)$

NAND gate truth table		
Input		Output
A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

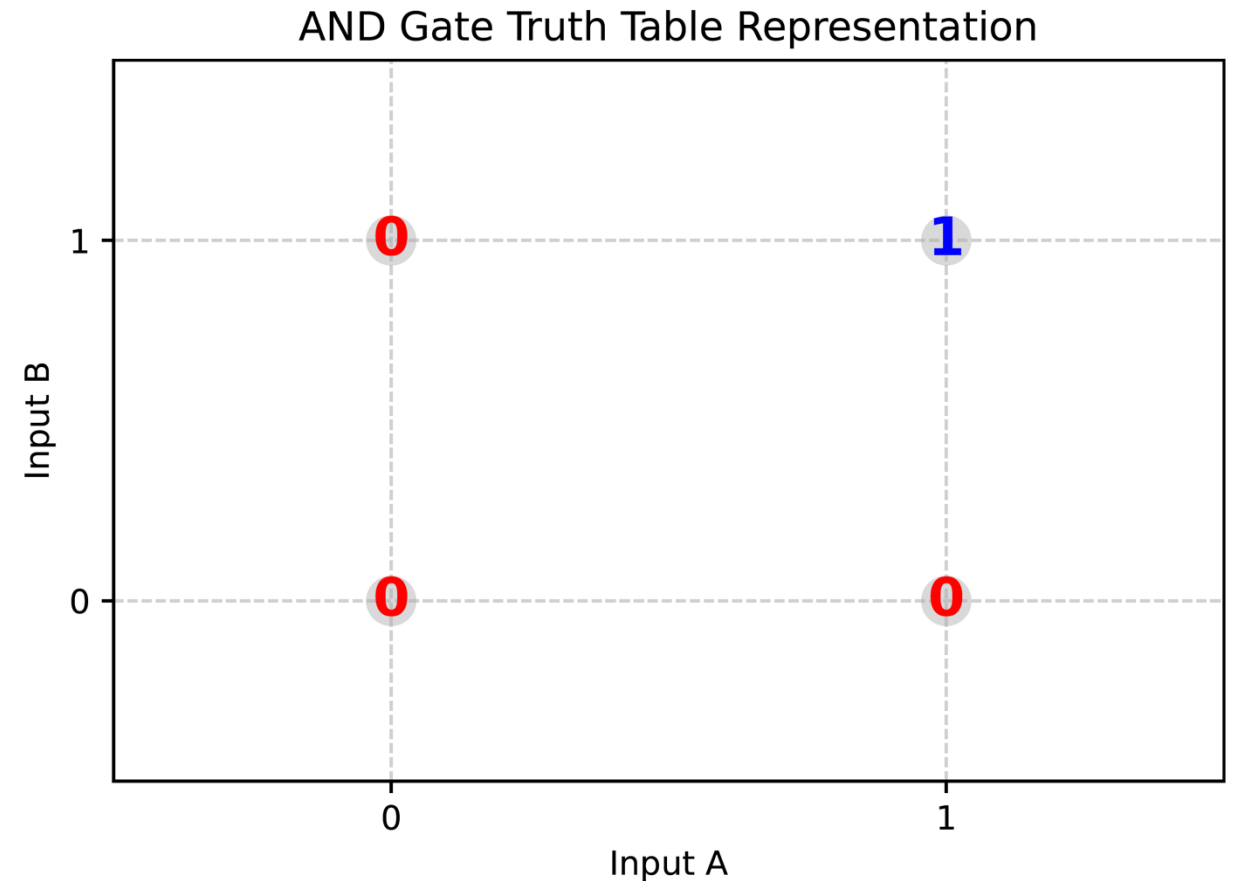
AND gate truth table		
Input		Output
A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

OR gate truth table		
Input		Output
A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Perceptron: AND

AND does have a linear boundary

Using our geometric understanding of the perceptron, what weights and bias would yield the right boundary?



Perceptron: AND

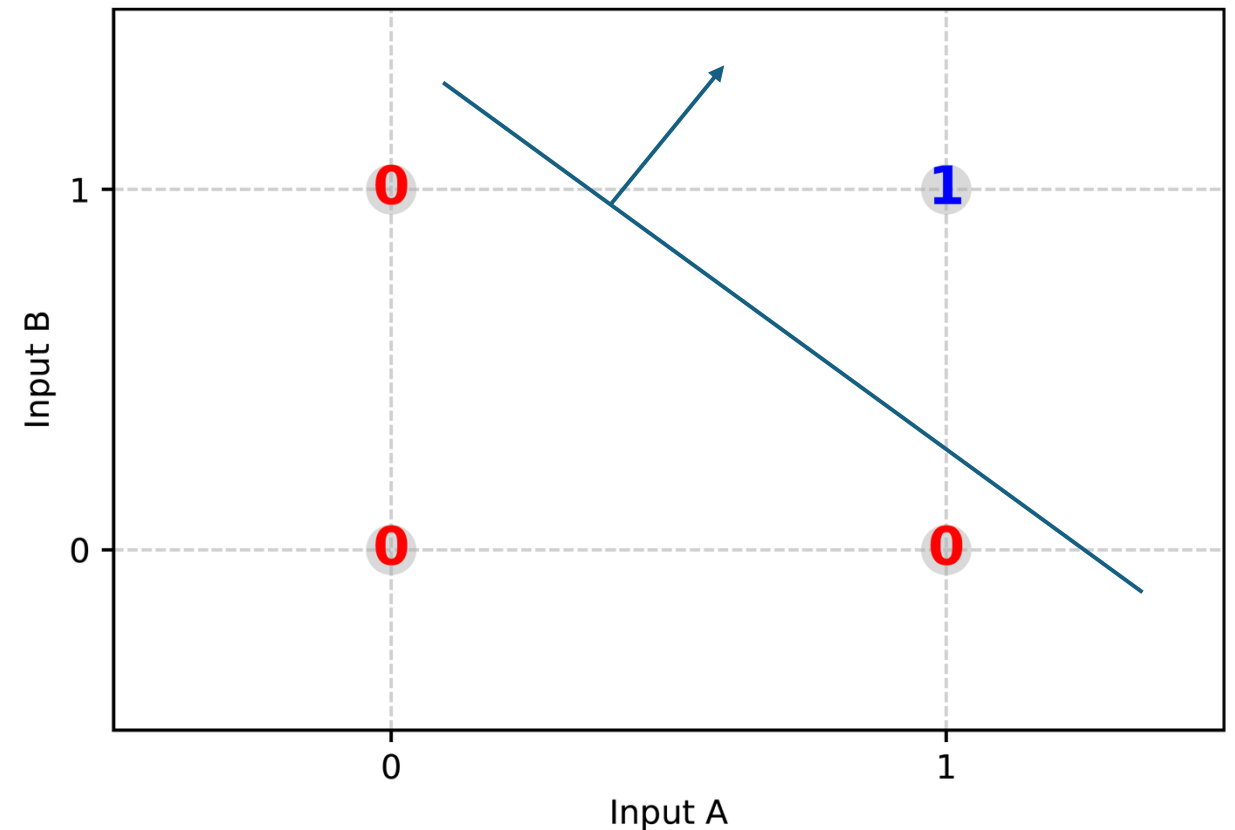
Weights = $[1, 1]$

Bias = $-\frac{3}{2}$

X	Y	.	+b	$\hat{y}$
0	0	0	-3/2	0
0	1	1	-1/2	0
1	0	1	-1/2	0
1	1	2	1/2	1

Bias intuition: a negative bias makes it *harder* for an example to be positive, moving the boundary in the positive direction of the weight vector

AND Gate Truth Table Representation

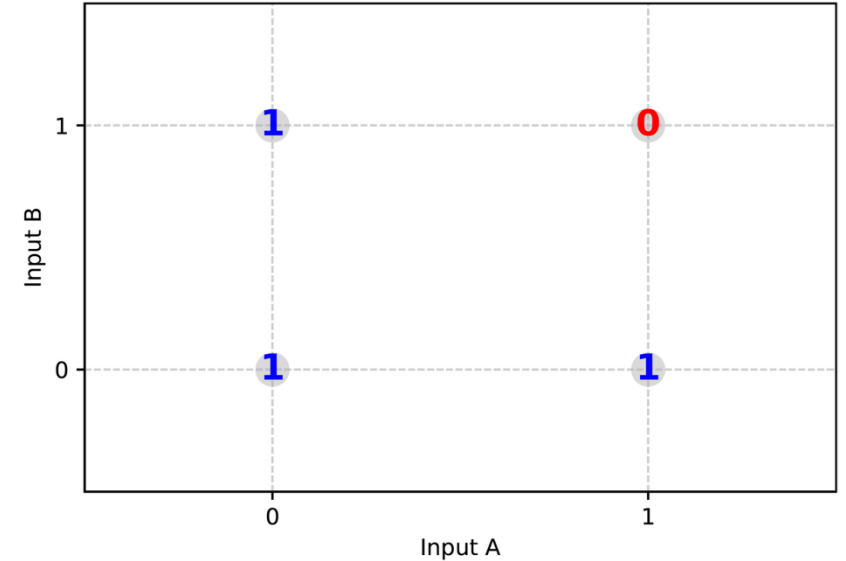


Perceptron: OR and NAND

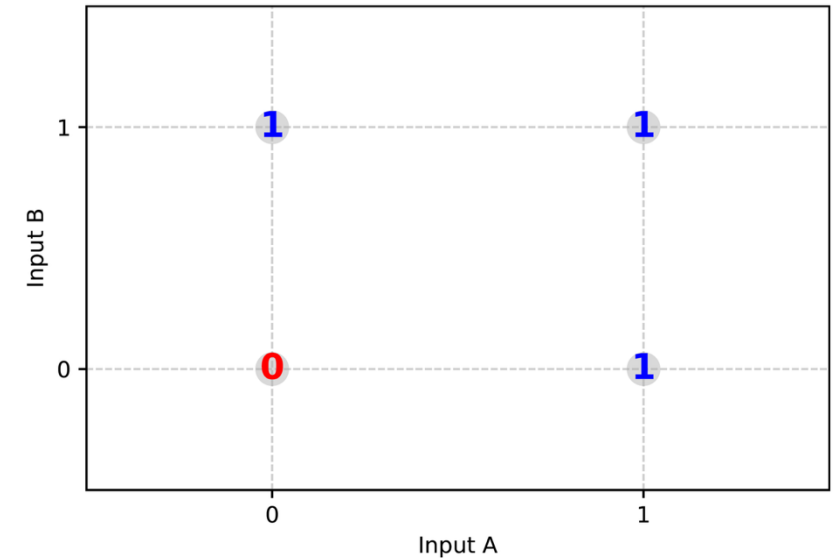
OR and NAND also have linear boundaries

Try and find weights which will classify these correctly

NAND Gate Truth Table Representation



OR Gate Truth Table Representation



Perceptron: OR and NAND

OR Gate

Weights = $[1, 1]$

Bias = $-\frac{1}{2}$

X	Y	$\cdot$	+b	$\hat{y}$
0	0	0	-1/2	0
0	1	1	1/2	1
1	0	1	1/2	1
1	1	2	3/2	1

NAND Gate

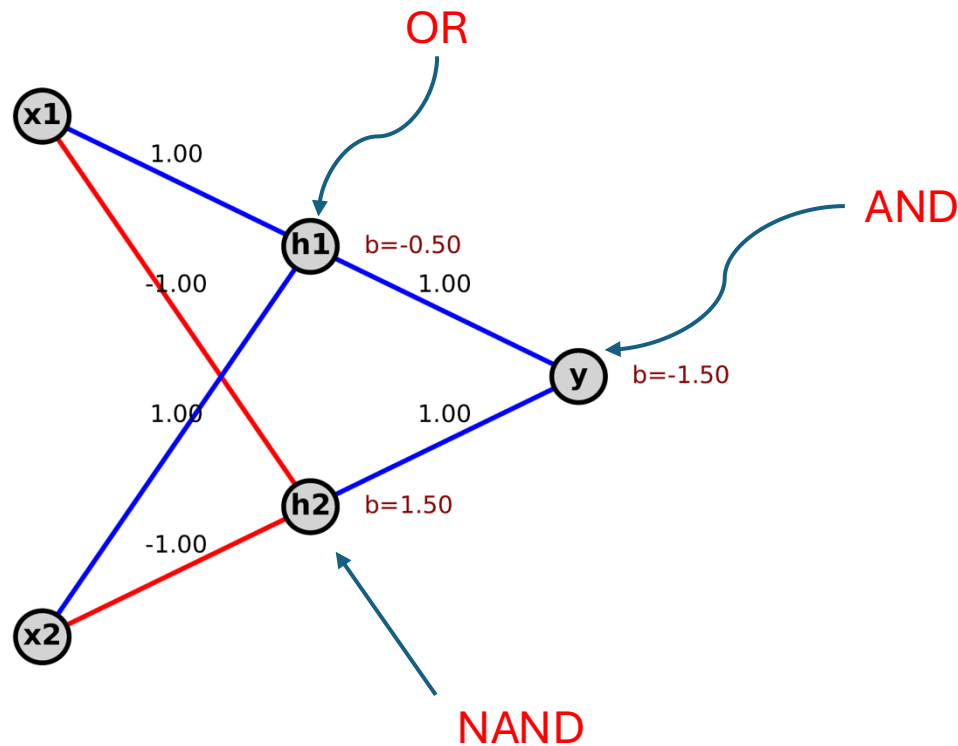
Weights = $[-1, -1]$

Bias = $3/2$

X	Y	$\cdot$	+b	$\hat{y}$
0	0	0	3/2	1
0	1	-1	1/2	1
1	0	-1	1/2	1
1	1	-2	-1/2	0

Perceptron: MLP

Putting it together, to solve the XOR problem we need combine these three perceptrons together



Non-Linear Functions

The MLP depends on some non-linear function $f(\mathbf{z})$ to combine perceptrons

$f(\mathbf{z})$ is referred to as an activation function

With this step, the output will no longer be a linear combination of inputs

→ Our decision boundary will no longer be linear

Need a Non-Linear Activation

$$\begin{aligned}z^0 &= \mathbf{w}^0 \cdot \mathbf{x} \\z^1 &= \mathbf{w}^1 \cdot \mathbf{z}^0\end{aligned}$$

$$z^0 = [b^0 \ w_1^0] \cdot [1 \ x_1]^T = b^0(1) + w_1^0(x_1) = b^0 + w_1^0 x_1$$

$$z^1 = [b^1 \ w_1^1] \cdot [1 \ z^0] = b^1(1) + w_1^1(z^0)$$

$$z^1 = b^1 + w_1^1(b^0 + w_1^0 x_1)$$

$$= [w_1^1 b^0 + b^1] + (w_1^1 w_1^0 x_1)$$


Bias Term


Weight

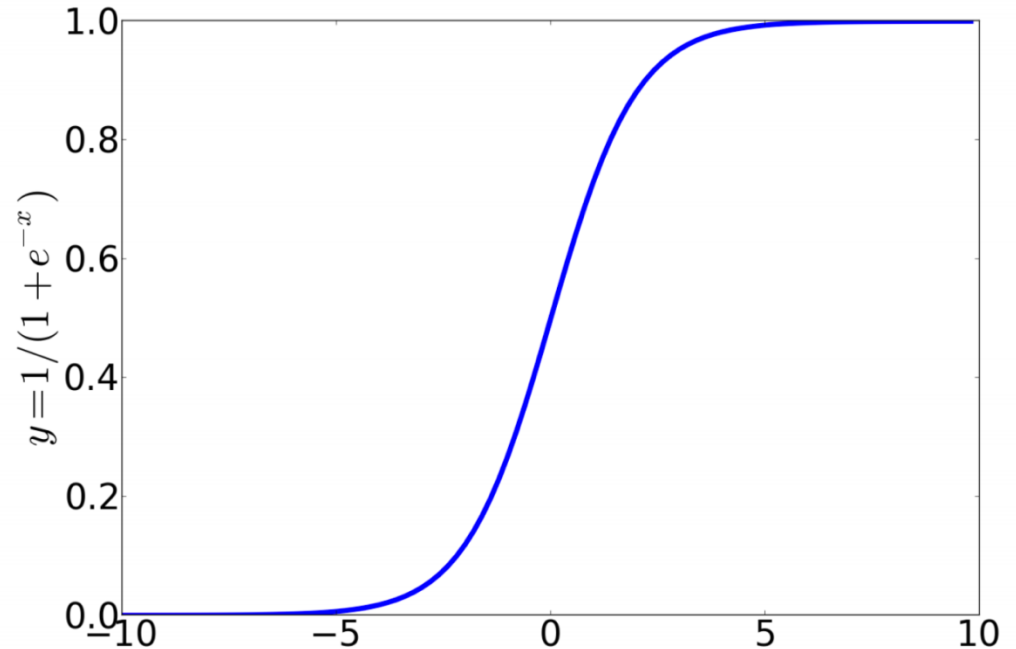
Reduces to a
single
perceptron

Common Activation Functions

Sigmoid

- Maps real numbers to (0,1)
- Used in logistic regression
- Differentiable!

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

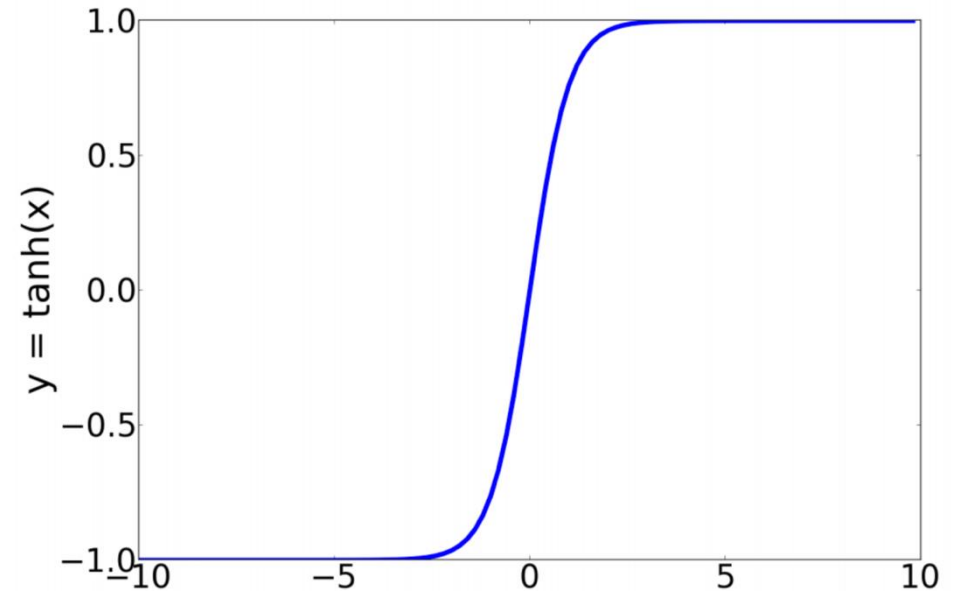


Common Activation Functions

Tanh

- Maps real numbers to (-1,1)
- More commonly used
- In practice, performs better than sigmoid

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

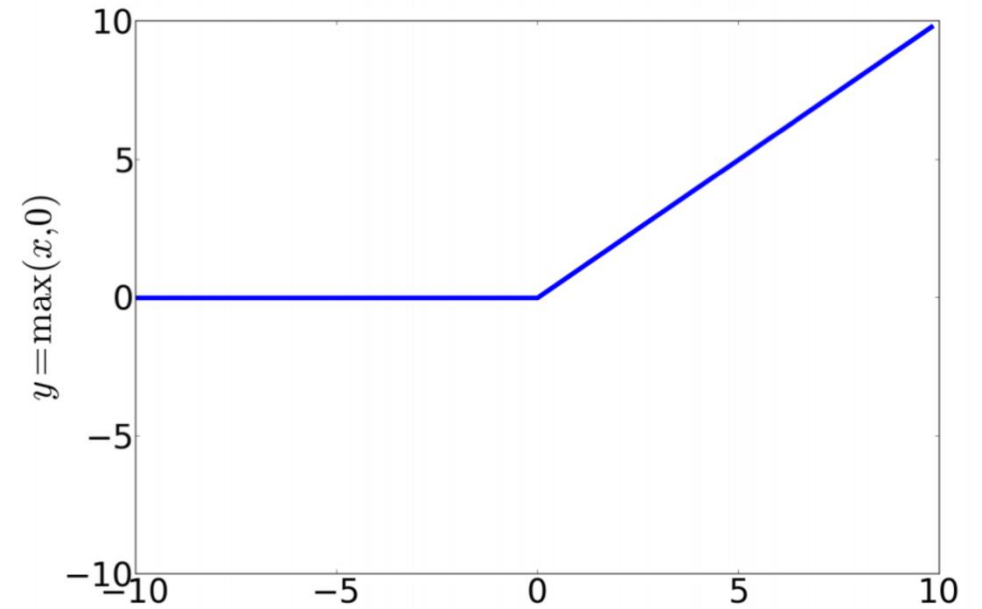


Common Activation Functions

ReLU

- Maps real numbers to $[0, \infty)$
- Simplest function, nearly linear

$$f(x) = \max(0, x)$$



Multi-Layer Perceptron

In our previous example, we had two inputs connected to two intermediate perceptrons

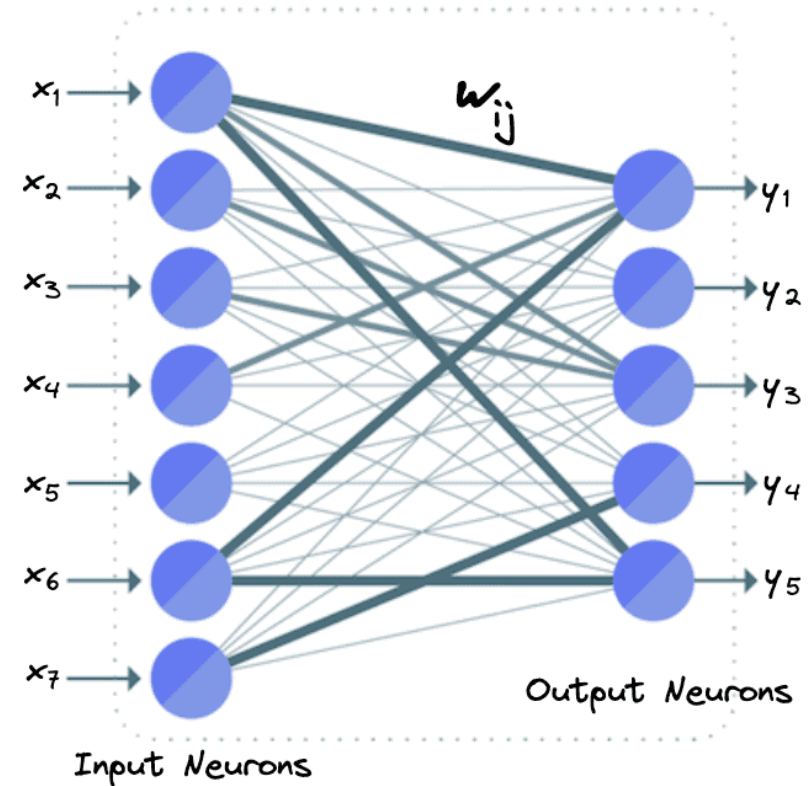
This is an example of a fully connected layer

Every input neuron is connected to every neuron in the next layer

Fully Connected Layers

Every input is connected to every output neuron

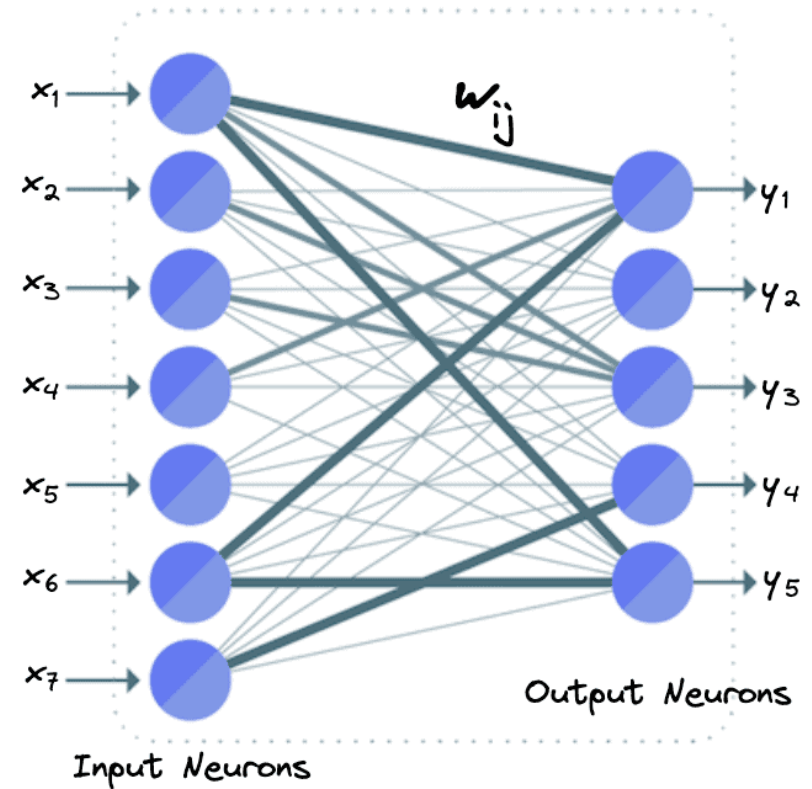
$$y_1 = \left(\sum_{j=0}^n w_{1j} x_j \right) + b_1$$



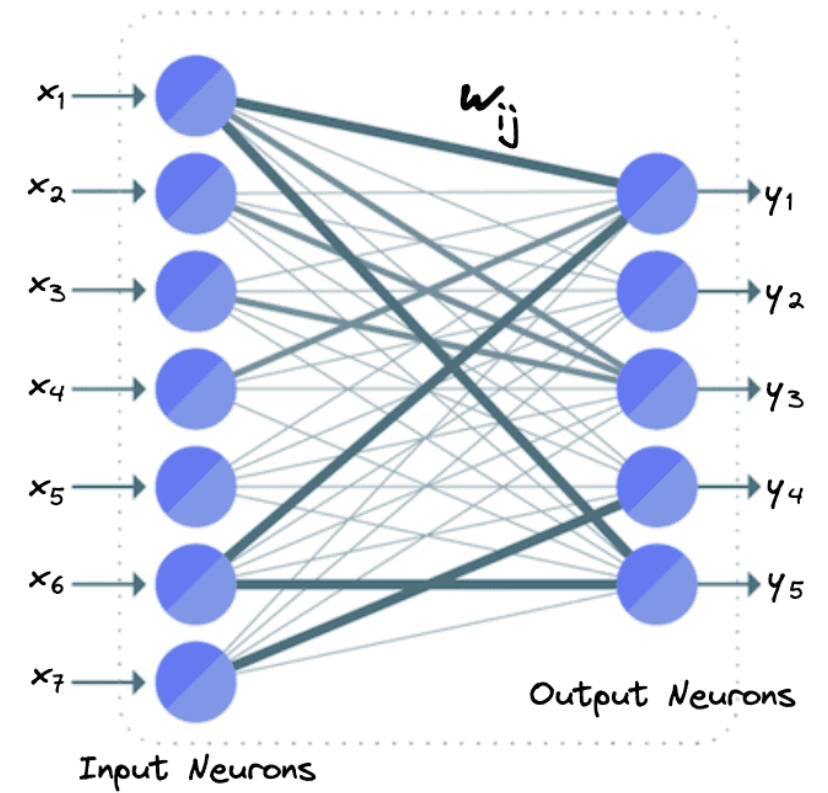
Fully Connected Layer

If the input vector $x \in \mathbb{R}^n$ and the output $y \in \mathbb{R}^m$, we'll need $m \times n$ weights to capture all of these dot products

Store these values in a weight matrix: $W \in \mathbb{R}^{m \times n}$



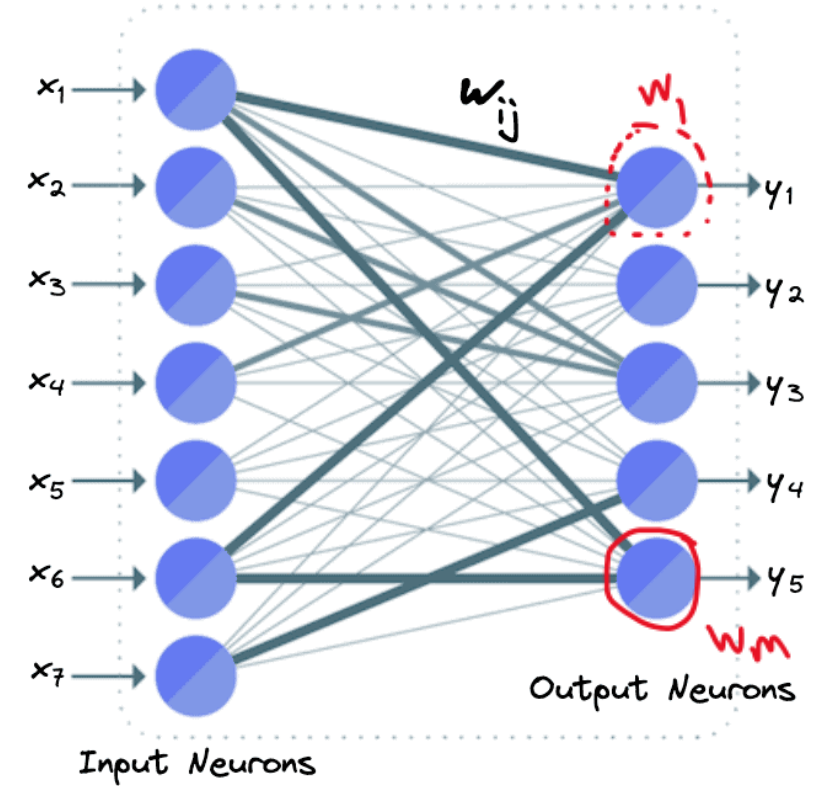
Fully Connected Layer



Fully Connected Layer

$$\sigma \left(\begin{bmatrix} W \end{bmatrix}_{m \times n} \begin{bmatrix} X \end{bmatrix}_{n \times 1} + \begin{bmatrix} B \end{bmatrix}_{m \times 1} \right) = \begin{bmatrix} Y \end{bmatrix}_{m \times 1}$$

$$\begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

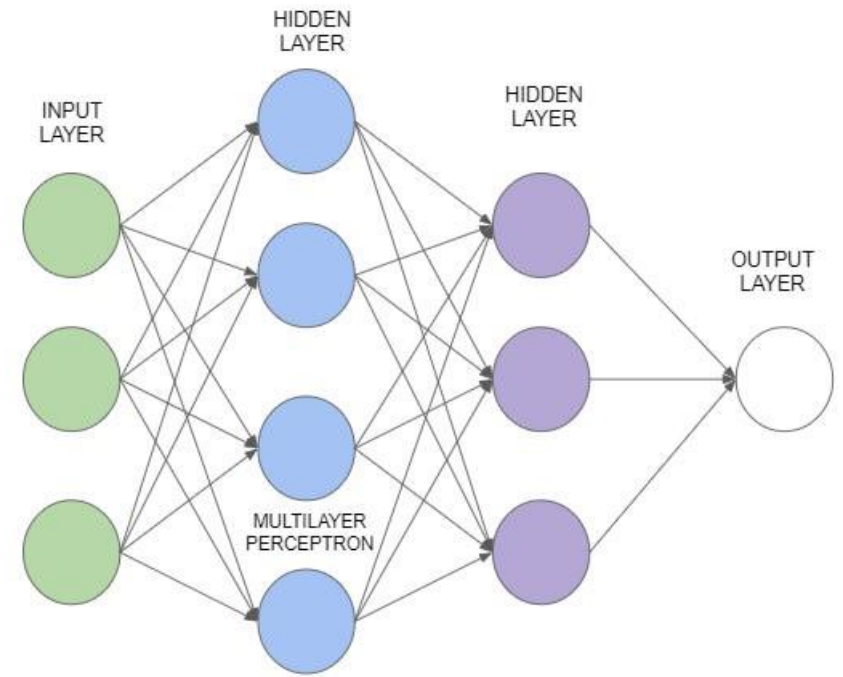


Multi-Layer Perceptron

A multi-layer perceptron is a series of fully connected layers

Intermediate layers are referred to as hidden layers

The final layer is called the output layer



Terminology

Parameters of a multilayer perceptron

- The learnable values that are updated during the training loop
- Weights, biases

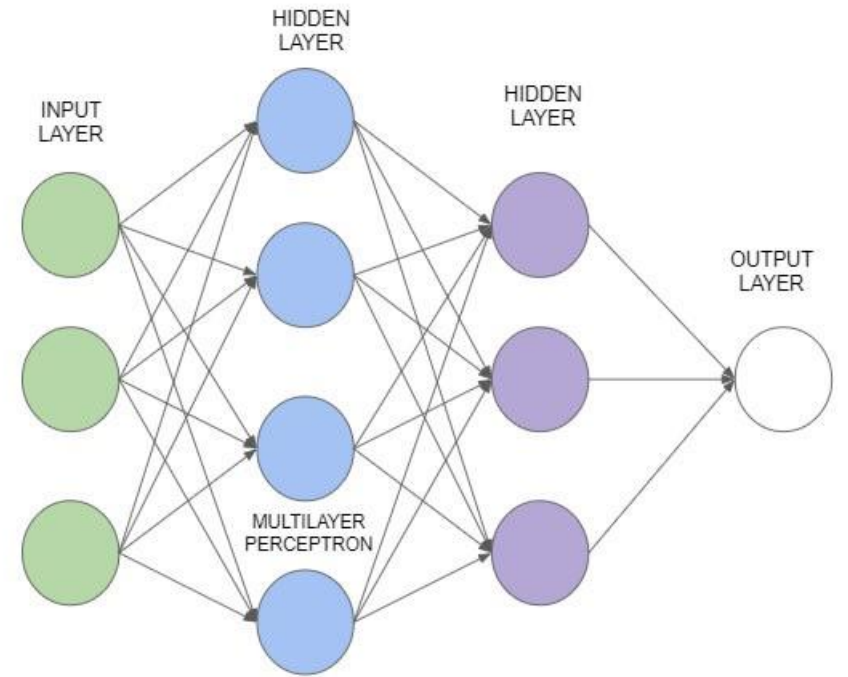
Hyper-parameters

- Configurable values that are provided to the model, and we change before training
- Number of layers, number of hidden units, the choice of the activation function

Parameters

Consider the following network

How many trainable parameters does it have?



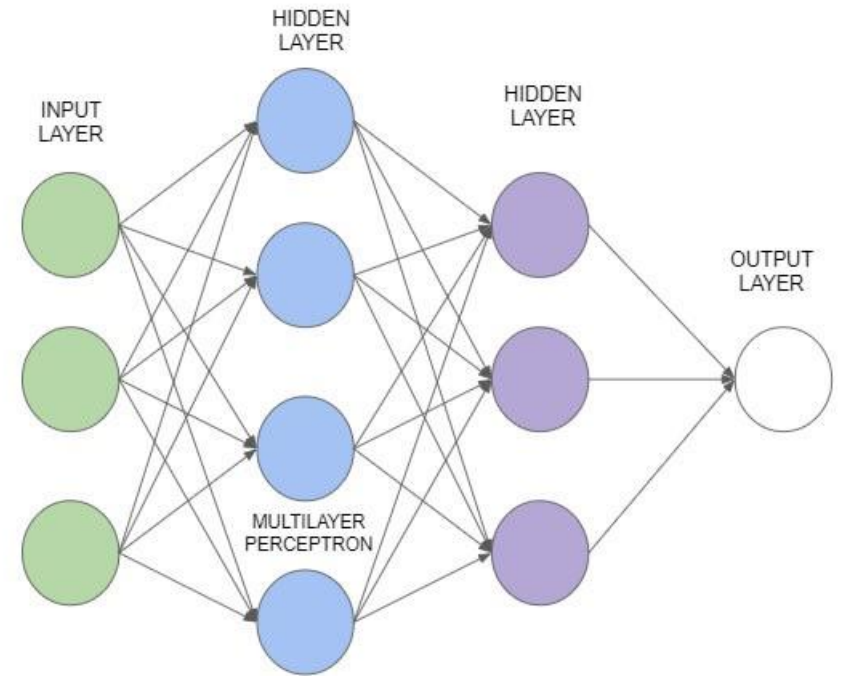
Parameters

Consider the following network

How many trainable parameters does it have?

$$\begin{aligned}W^1 &\in \mathbb{R}^{4 \times 3}, B^1 \in \mathbb{R}^4 \\W^2 &\in \mathbb{R}^{3 \times 4}, B^2 \in \mathbb{R}^3 \\W^3 &\in \mathbb{R}^{3 \times 1}, B^3 \in \mathbb{R}\end{aligned}$$

$$\text{Params} = (12 + 4) + (12 + 3) + (3 + 1) = 35$$



MLP for Classification

The single output node is fine for binary classification

What about multi-class problems?

MLP for Classification

We can increase the size of the output layer to match the number of classes in our problem

The outputs of the neural network in their raw form are called *logits*

We would like to turn these logits into probabilities

Softmax

Can be used for multi-class classification

Given a vector $\mathbf{z} \in \mathbb{R}^n$, $\text{softmax}(\mathbf{z})_i = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$

The values outputted by the softmax function are all between 0 and 1

They add up to 1

→ Probability distribution over classes

Neural Language Models

How can we model $P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$

Consider a bigram model – our input will be a single token, and the output will also be a single token

We need to represent these tokens as vectors so they can be inputted into our neural model

Neural Language Models

Naïve option:

- Map each token to an arbitrary number

- Vectorize as a 1-hot vector

Issues:

- We aren't encoding any information with this approach

- If two tokens map to neighboring indices, this doesn't capture anything

Neural Language Model

A better approach: Word Embeddings

Dense representations, where their location in the vector spaces encodes some meaning

Pretrained: can leverage training which has occurred before I train my model

Neural Language Models: In Practice

In practice, we first generate a vocabulary using our training data

Each token is assigned an index and corresponding 1-hot vector

This 1-hot vector is multiplied with a dense embedding layer initialized from our word embeddings

- Acts as a “lookup” in our embedding matrix

What is the output space?

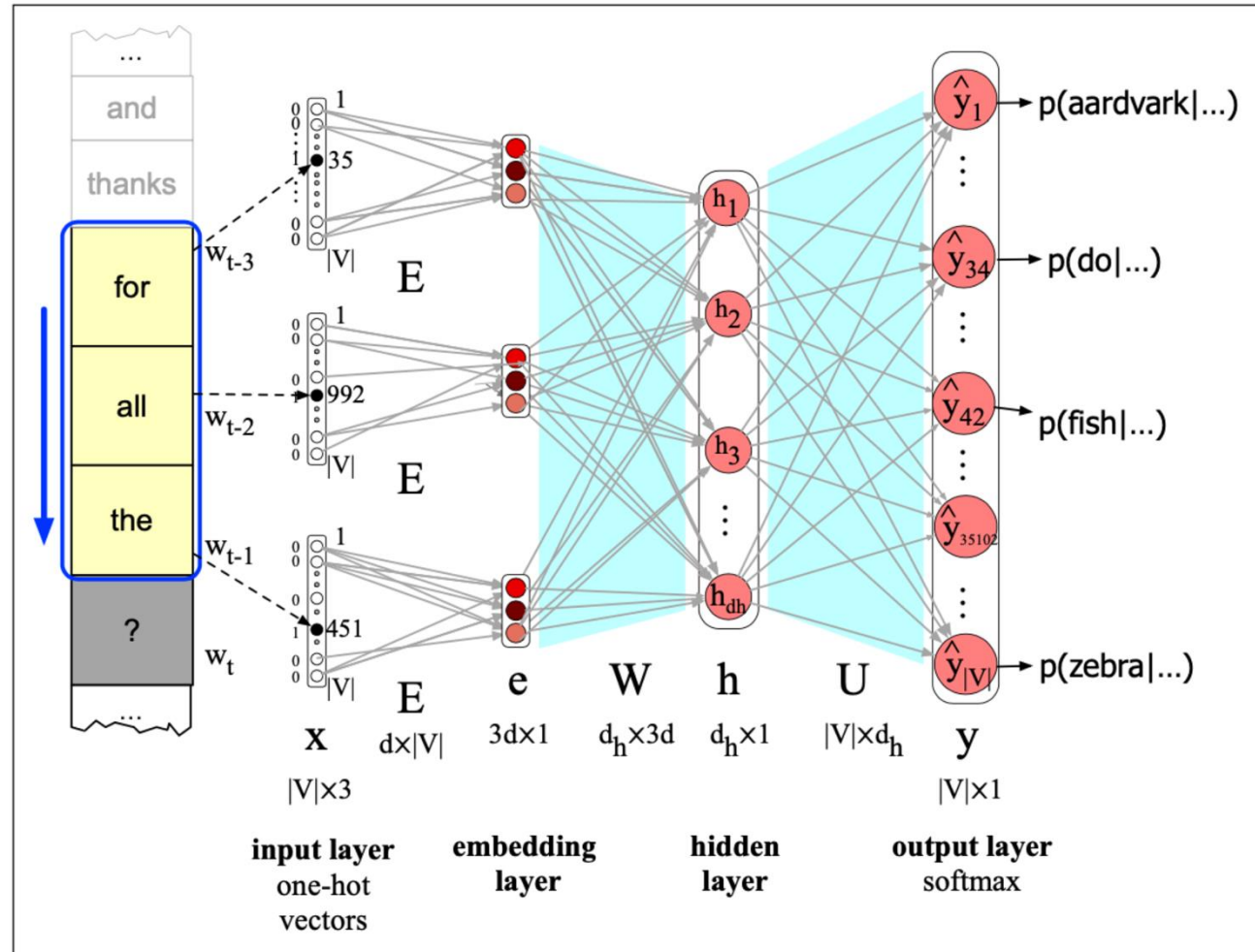
Neural Language Models: In Practice

$$\mathbf{e} = [\mathbf{Ex}_{t-3}; \mathbf{Ex}_{t-2}; \mathbf{Ex}_{t-1}]$$

$$\mathbf{h} = \sigma(\mathbf{We} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{Uh}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$



How do we get the weights?

Weights and biases are initialized randomly

We would expect that the model outputs will be fairly poor to start with

To improve, we should have some notion of *how poor* our model outputs are

If we have some function which models this cost, we can minimize it

Minimizing the errors, means improving the model

This function is referred to as the *loss function*

Cross Entropy Loss

CrossEntropyLoss

```
CLASS torch.nn.CrossEntropyLoss(weight=None, size_average=None, ignore_index=-100, reduce=None,  
reduction='mean', label_smoothing=0.0) \[SOURCE\]
```

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_{y_n} \log \frac{\exp(x_{n,y_n})}{\sum_{c=1}^C \exp(x_{n,c})} \cdot 1\{y_n \neq \text{ignore_index}\}$$

Weight Updates

Now that we have a cost function, we can use it to improve our weights

Main idea in neural networks:

1. Input an example and get the predicted model output
2. Compare the predicted output/distribution to the true output
3. Calculate how to change every weight to slightly minimize my cost



Gradient Descent

Calculating Gradients

Most neural networks have a large number of parameters

We can calculate how much of an effect each individual parameter has on the final prediction, and then modify that weight in the direction which minimizes our cost

To find how much the loss changes with respect to a weight, we calculate the derivative – this will be the focus of next week!

Calculating Gradients - Example

First, recall the chain rule of derivation:

$$f(x) = f(u(v(x))),$$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f}{\partial u} \frac{\partial u}{\partial v} \frac{\partial v}{\partial x}$$

Consider the following function:

$$L(a, b, c) = c * (2a + b)$$

Calculating Gradients - Example

Break up this function into smaller ones:

$$L(a, b, c) = c * (2b + a)$$

$$d = 2 * b$$

$$e = d + a$$

$$L = c * e$$

What is $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, $\frac{\partial L}{\partial c}$?

Calculating Gradients - Example

Break up this function into smaller ones:

$$L(a, b, c) = c * (2b + a)$$

$$d = 2 * b$$

$$e = d + a$$

$$L = c * e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}, \quad \frac{\partial L}{\partial c} = e$$

Calculating Gradients - Example

$$L(a, b, c) = c * (2b + a)$$

$$d = 2 * b$$

$$e = d + a$$

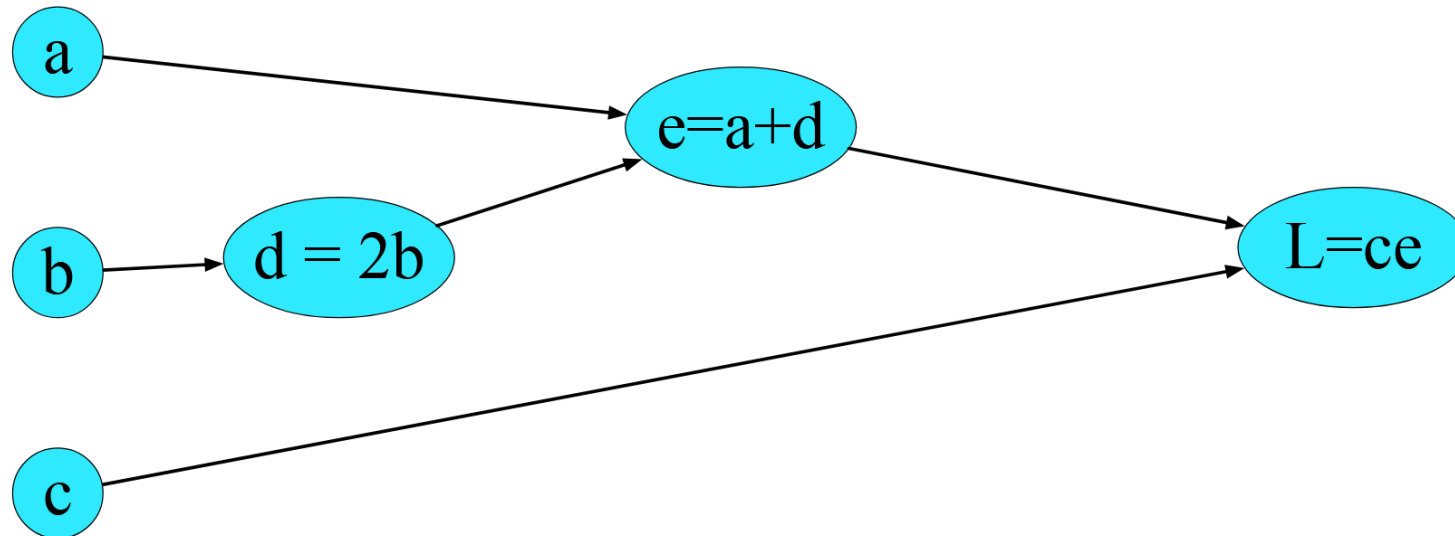
$$L = c * e$$

Calculate the partial derivatives:

$$\frac{\partial L}{\partial c}, \frac{\partial L}{\partial e}, \frac{\partial e}{\partial d}, \frac{\partial e}{\partial a}, \frac{\partial d}{\partial b}$$

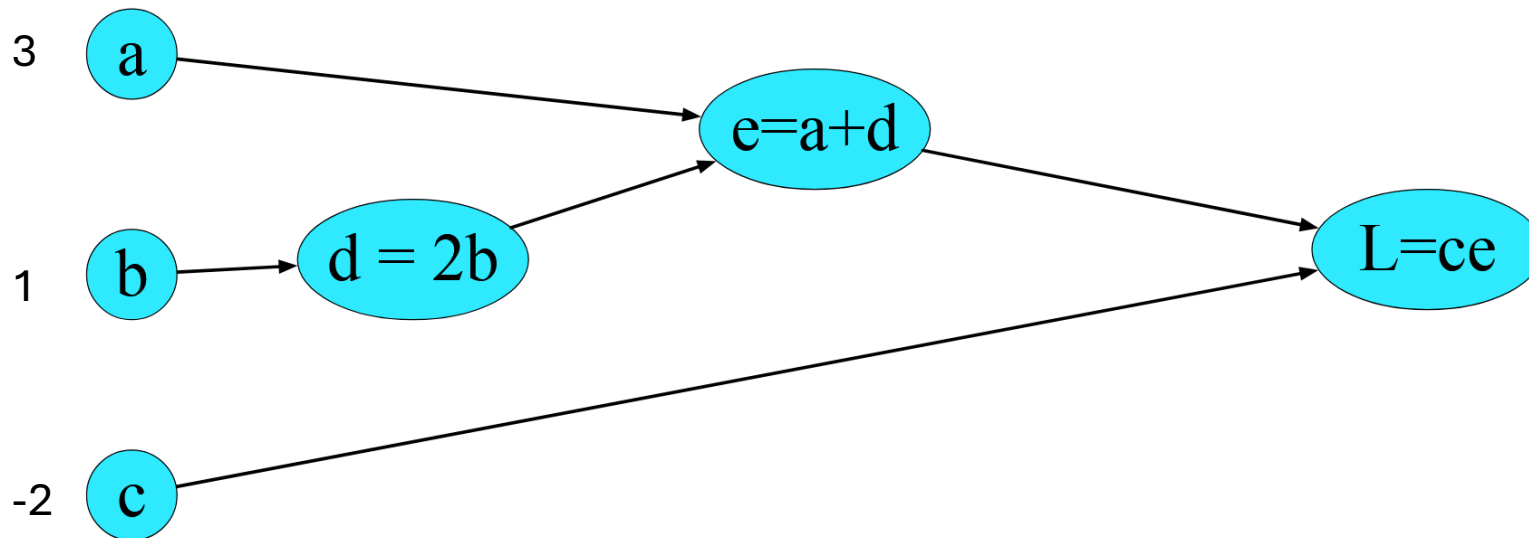
Computation Graphs

Can model these functions and compositions in a computation graph:



Computation Graphs

Can model these functions and compositions in a computation graph:



Computation Graphs

$$L(a, b, c) = c * (2b + a)$$

Partials can be used for automatic differentiation

$$\begin{aligned}d &= 2 * b \\e &= d + a \\L &= c * e\end{aligned}$$

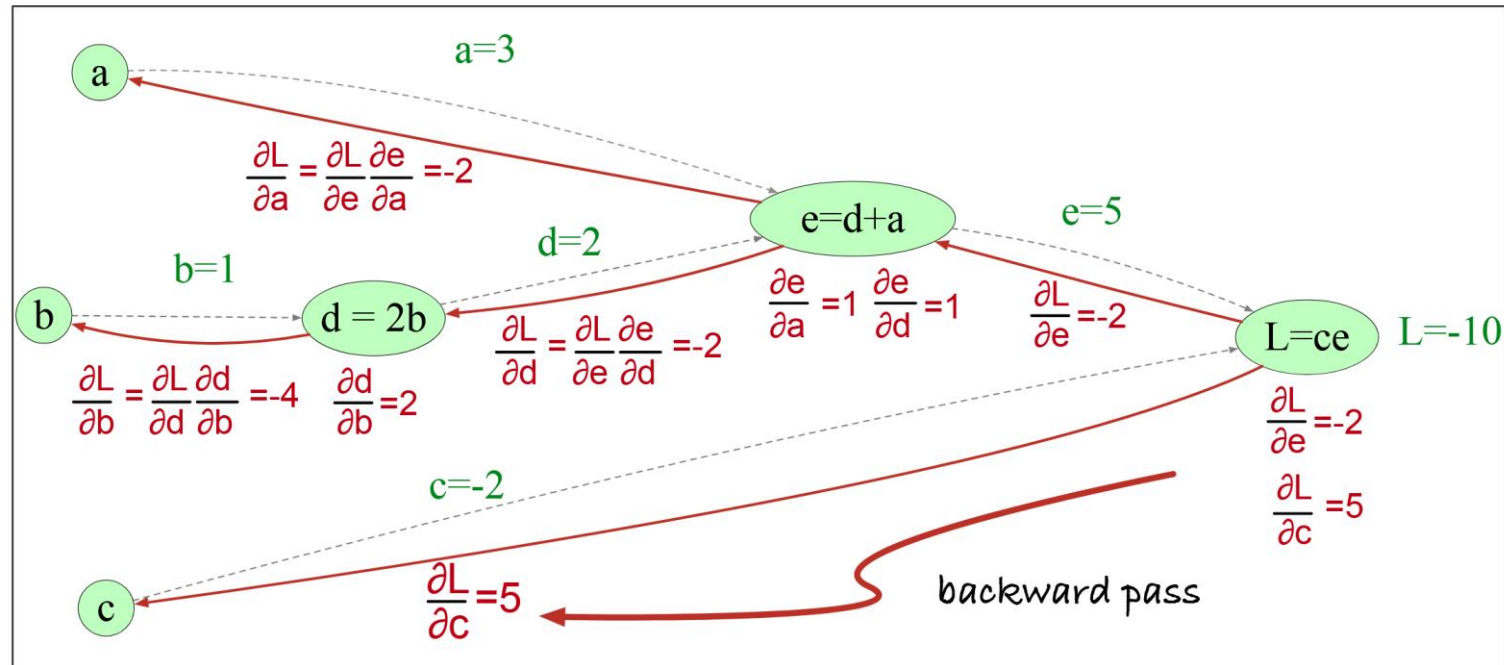
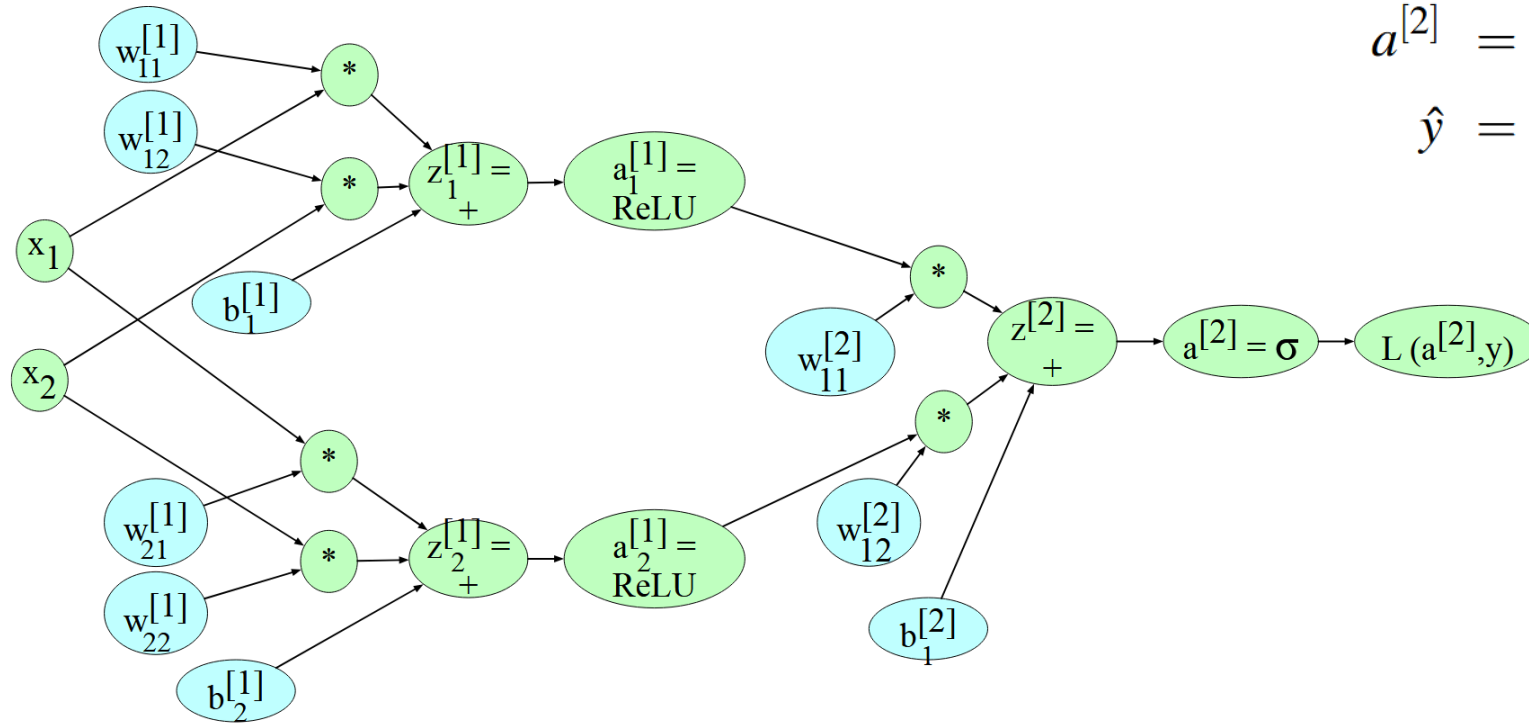


Figure 7.14 Computation graph for the function $L(a, b, c) = c(a + 2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.

Graphs for Neural Networks



$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]})$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

$$\mathbf{a}^{[2]} = \sigma(\mathbf{z}^{[2]})$$

$$\hat{y} = \mathbf{a}^{[2]}$$